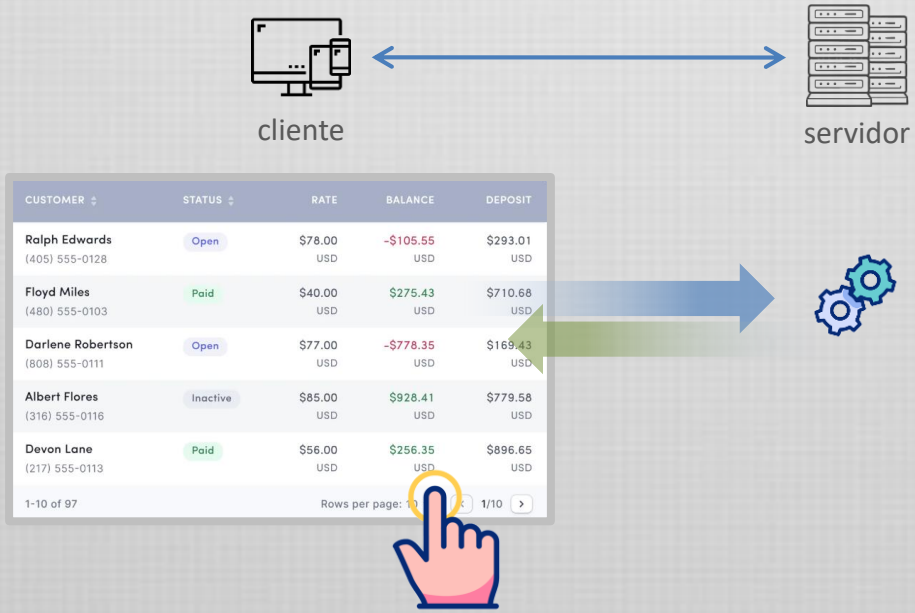
Ingeniería de Aplicaciones Web

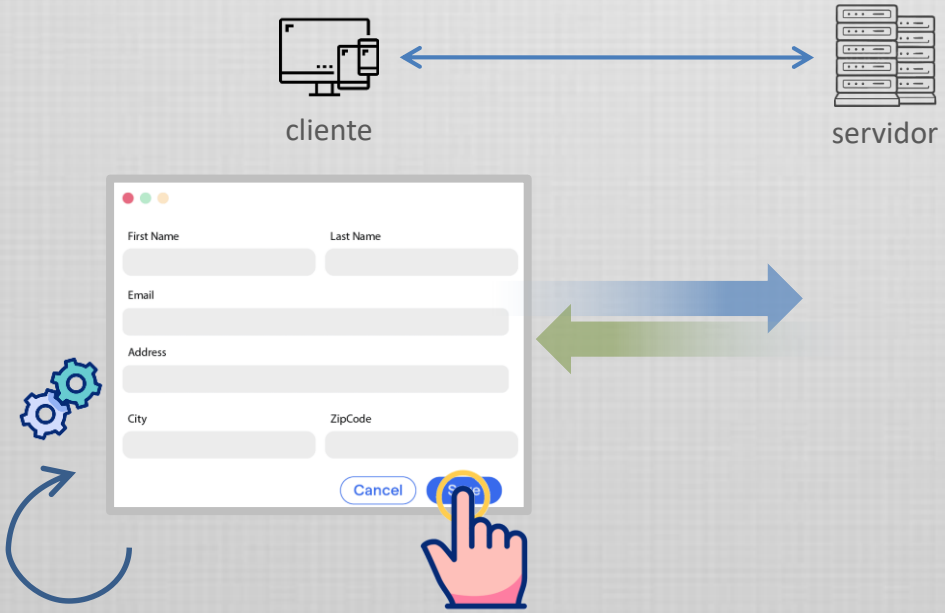
Diego C. Martínez

Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur

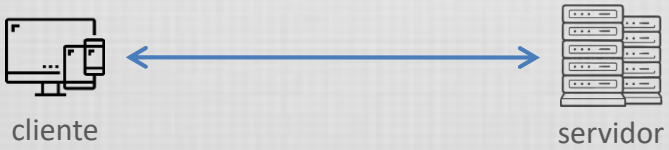
Formularios



Formularios



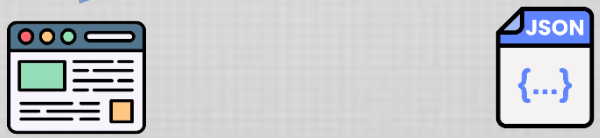
Formularios



Un servidor puede servir páginas web (HTML, CSS, etc)

... pero también puede servir datos (JSON, XML,etc)

http://unsitio.com/abc/def/123



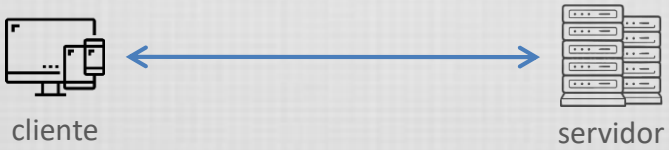
(una URL devuelve una cosa o la otra)

http://unsitio.com/abc/def/123

La idea es tener una forma de

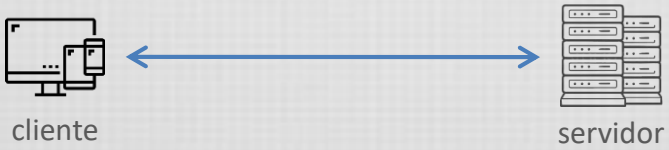
- Proveer datos para desacoplar **presentación** de **administración** de datos en una aplicación web
- Integrar datos en diversas plataformas
- Particionar aplicaciones complejas en componentes independientes

APIs



En una página web, las URLs indican lo que estamos viendo en la interfaz

En un servicio web, las URLs indican recursos para obtener o manipular



Application Path

http://unsitio.com/home

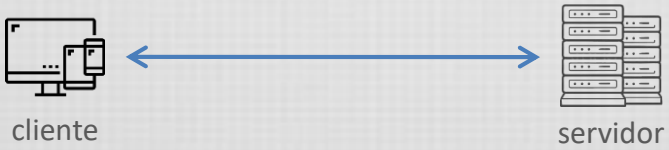
http://unsitio.com/productos

http://unsitio.com/contactos

http://unsitio.com/promociones

Las URL que representan páginas
definen las vistas de la UI
Se ven en la barra de direcciones
del navegador

APIs



API

Cuando operamos con datos, las URL definen los puntos de contacto (endpoints)

La API es una colección de endpoints

http://unsitio.com/productos

http://unsitio.com/ventas/argentina/clientes

http://unsitio.com/envios?d=2021&h=2026

http://unsitio.com/ofertas/visa

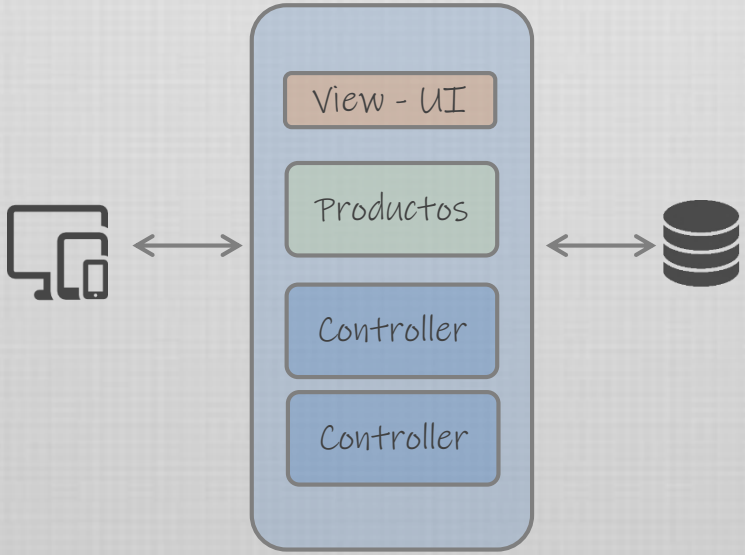
http://unsitio.com/ofertas/mastercard

Arquitecturas Monolíticas

Nuestro proyecto podría estar hecho íntegramente en Next
en un solo repositorio, en un solo deploy, toda la aplicación

Simplicidad

Desarrollo
Testing
Deployment
Escalabilidad vertical

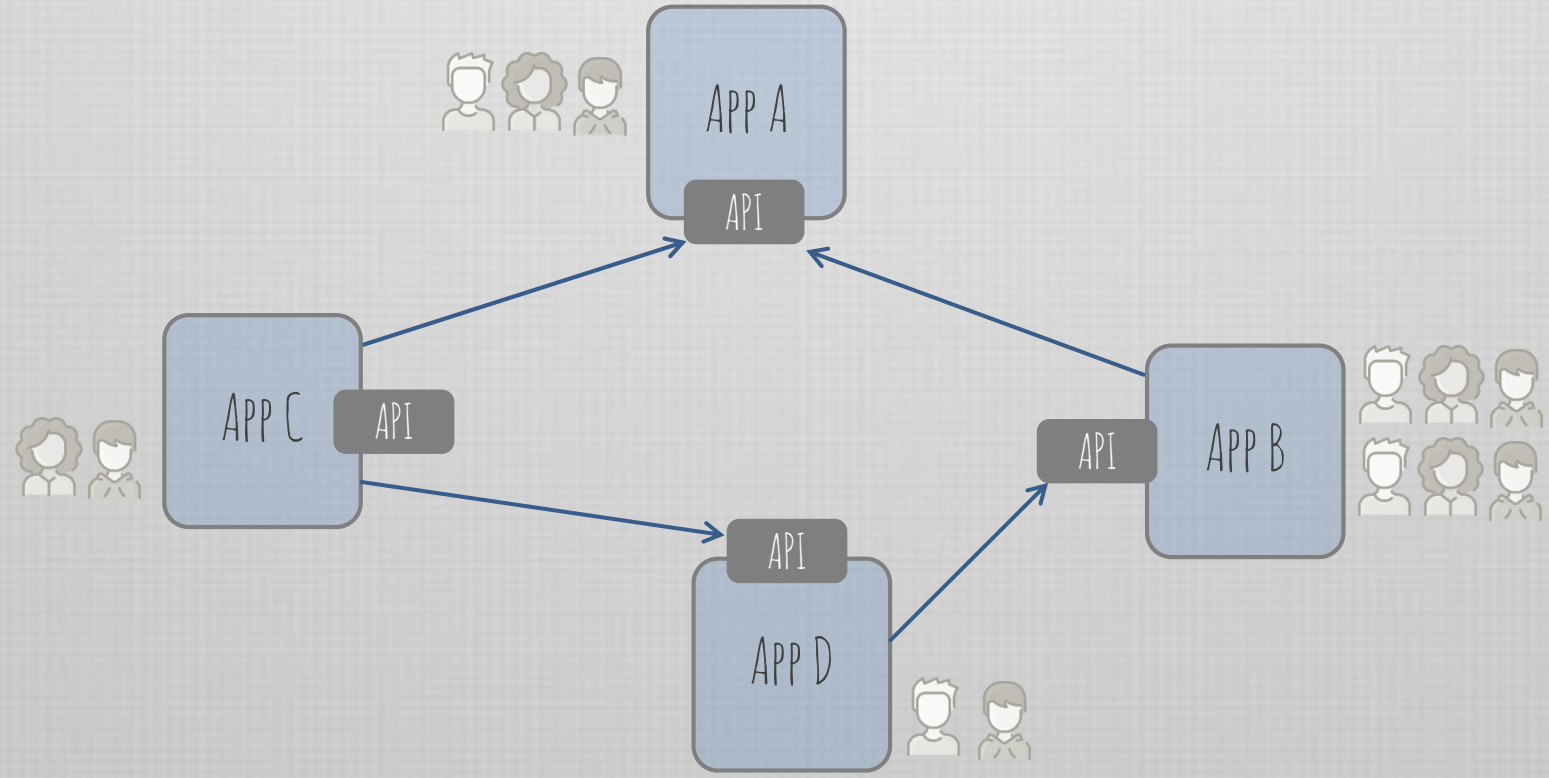


Dificultades

Desarrollo Colaborativo
STACK Tecnológico
Mantenimiento
Escalabilidad horizontal

Arquitecturas Monolíticas

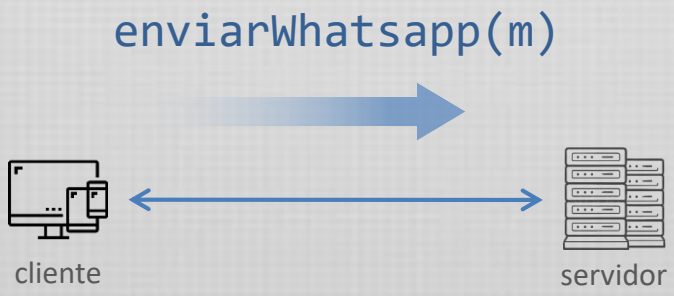
Pero si lo particionamos en diferentes aplicaciones,
obtenemos los mismos beneficios modulares que en el software



Esto es la esencia de los microservicios

APIs basadas en funciones

Permite a los clientes ejecutar comandos en servidores remotos como si estuviesen invocando funciones locales



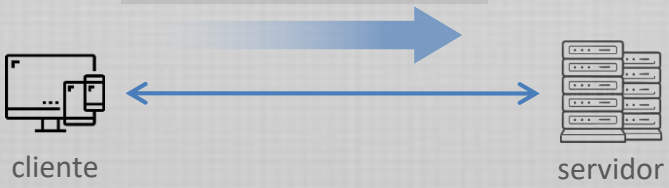
XML-RPC, JSON-RPC, SOAP, gRPC son ejemplos de este tipo de APIs

APIs basadas en consultas (queries)

Diseñadas para mayor flexibilidad en la especificación de los datos requeridos

Los clientes envían consultas estructuradas

```
query{
 countries{
  name
  continent{
    name
  }
 }
}
```



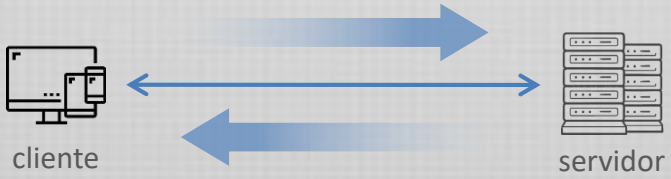
GraphQL es un ejemplo de este tipo de APIs

APIs basadas en eventos

Diseñadas para gestionar información en tiempo real o comunicaciones asincrónicas entre el cliente y el servidor

Websockets es un ejemplo de este tipo de APIs

También algunos servicios de mensajería como RabbitMQ o Kafka



La comunicación es bidireccional y de larga duración

Tipos de APIs

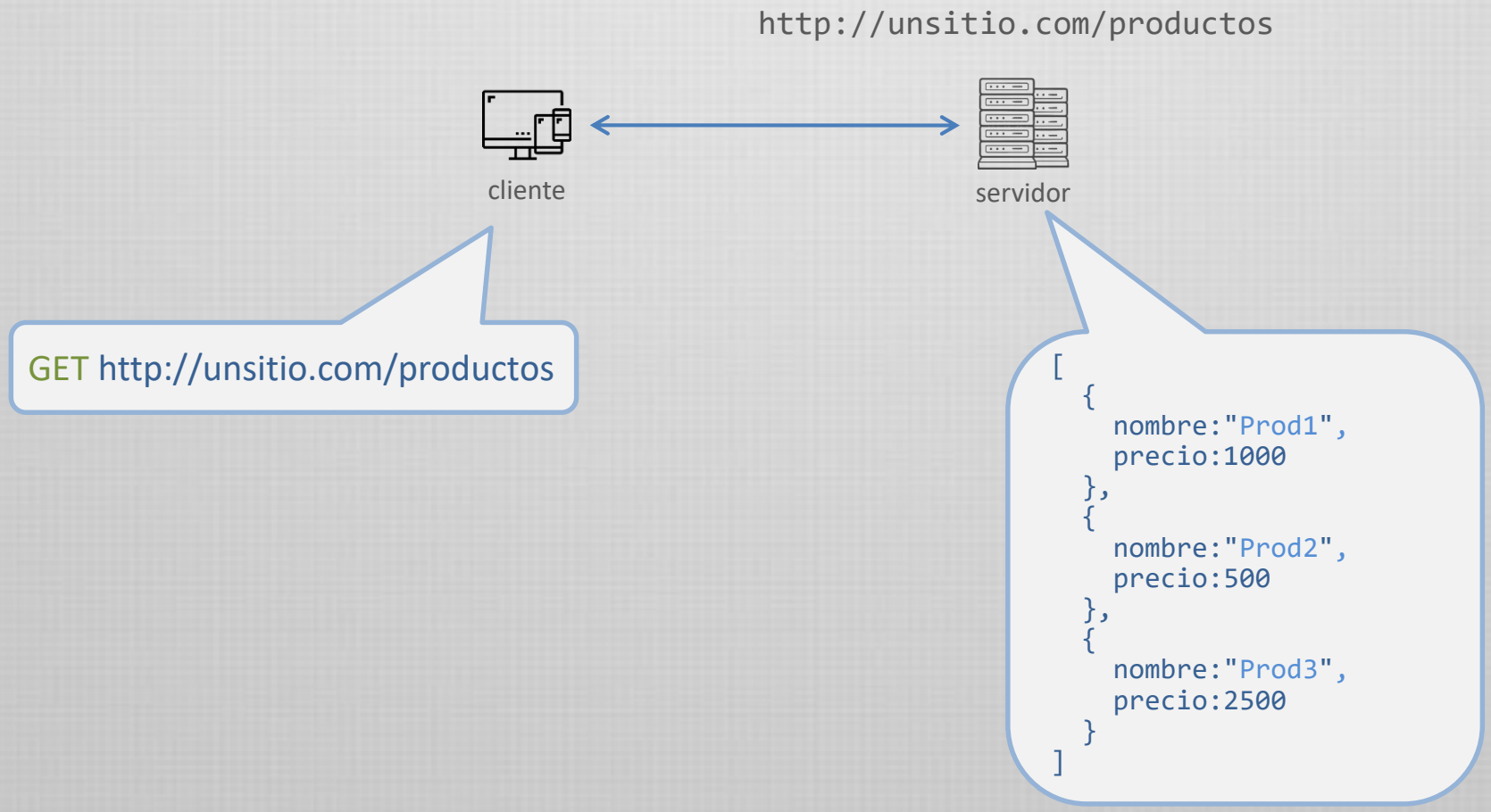
APIs basadas en recursos

Diseñadas para interactuar con objetos o entidades (recursos) en la web utilizando sus protocolos estándar (HTTP)

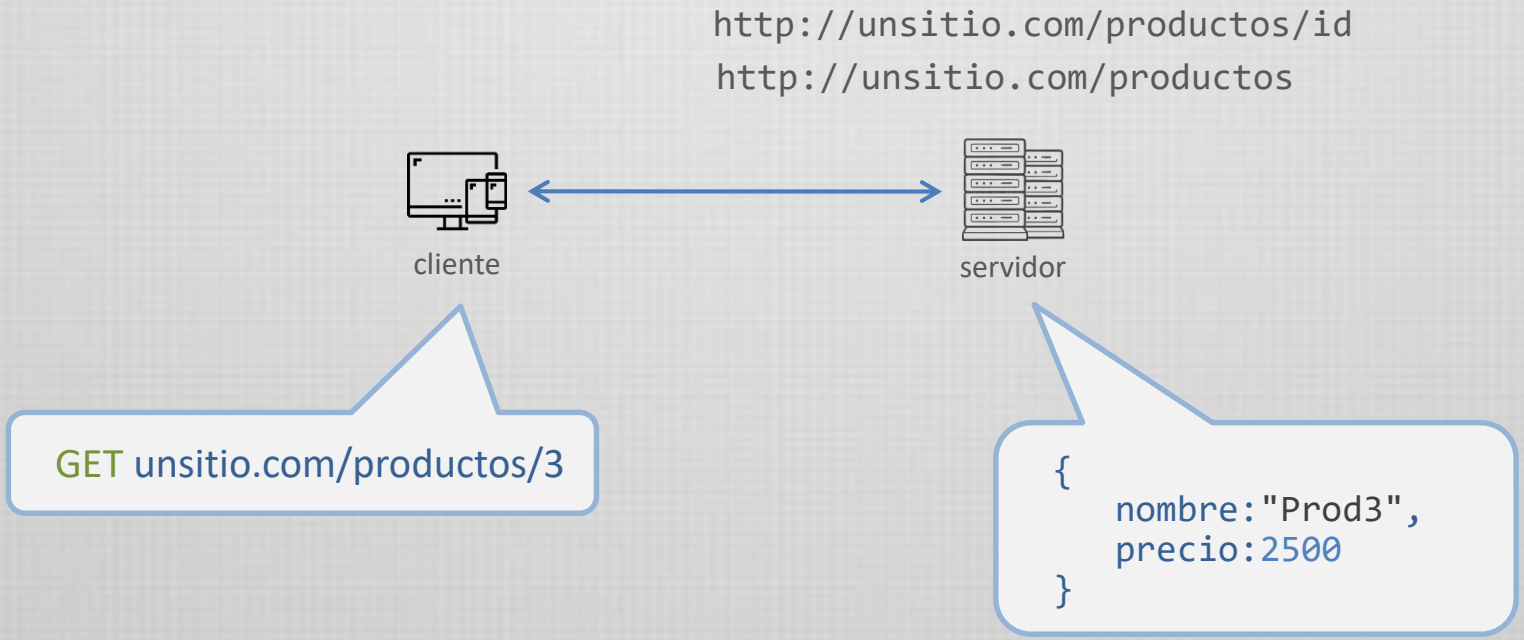
REST es el más común de este tipo de APIs

No requiere infraestructura especial

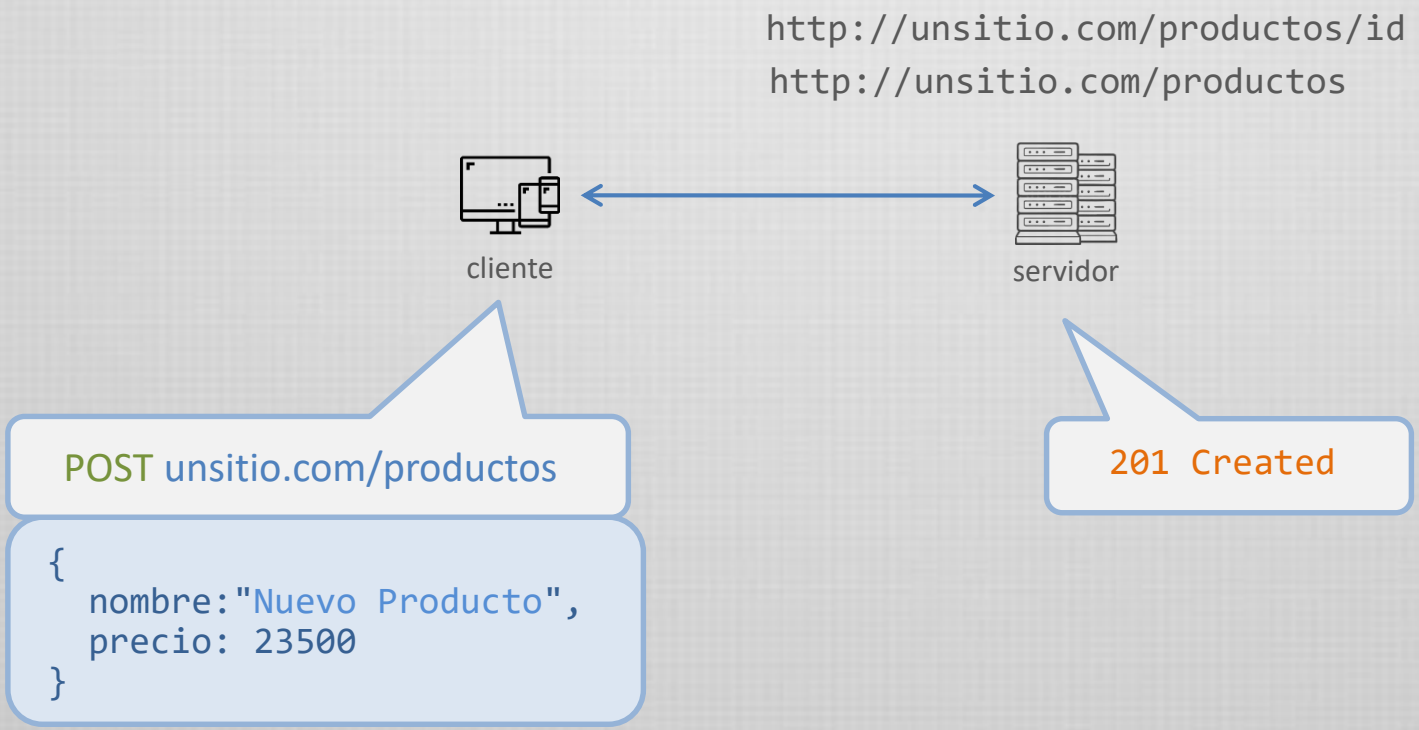
APIs basadas en recursos



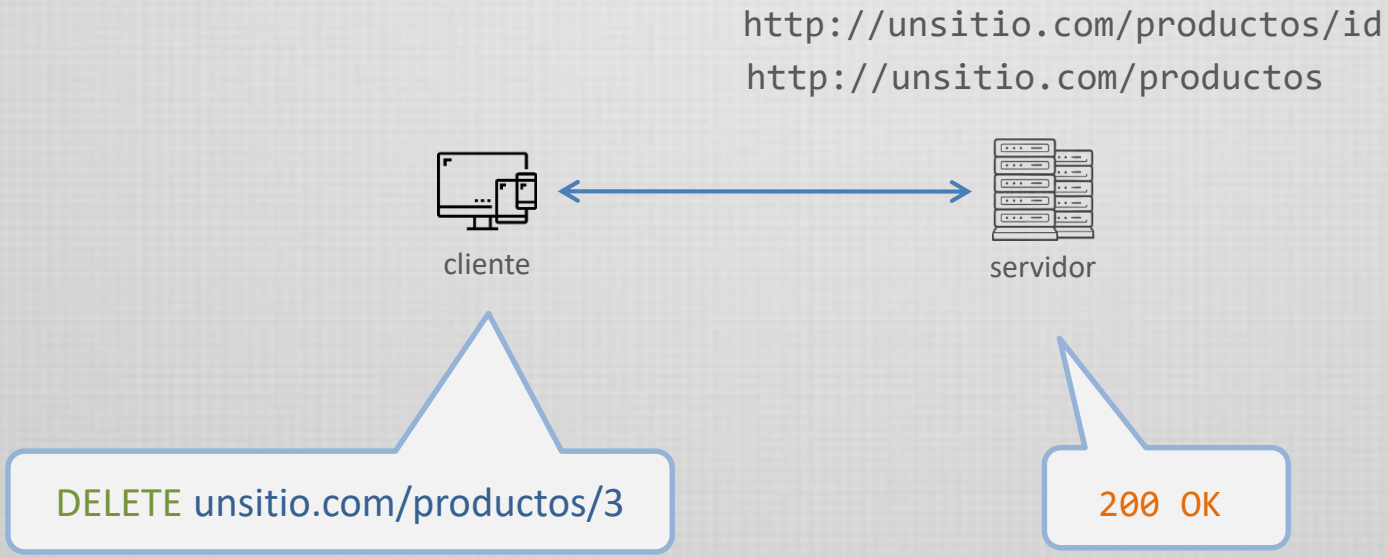
APIs basadas en recursos



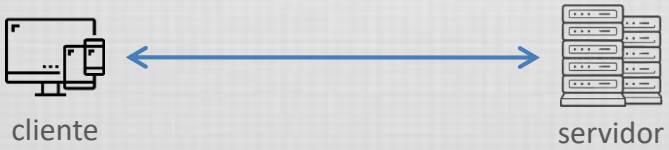
APIs basadas en recursos



APIs basadas en recursos

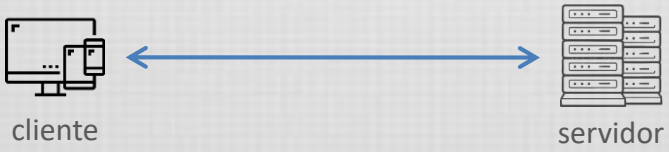


APIs basadas en recursos



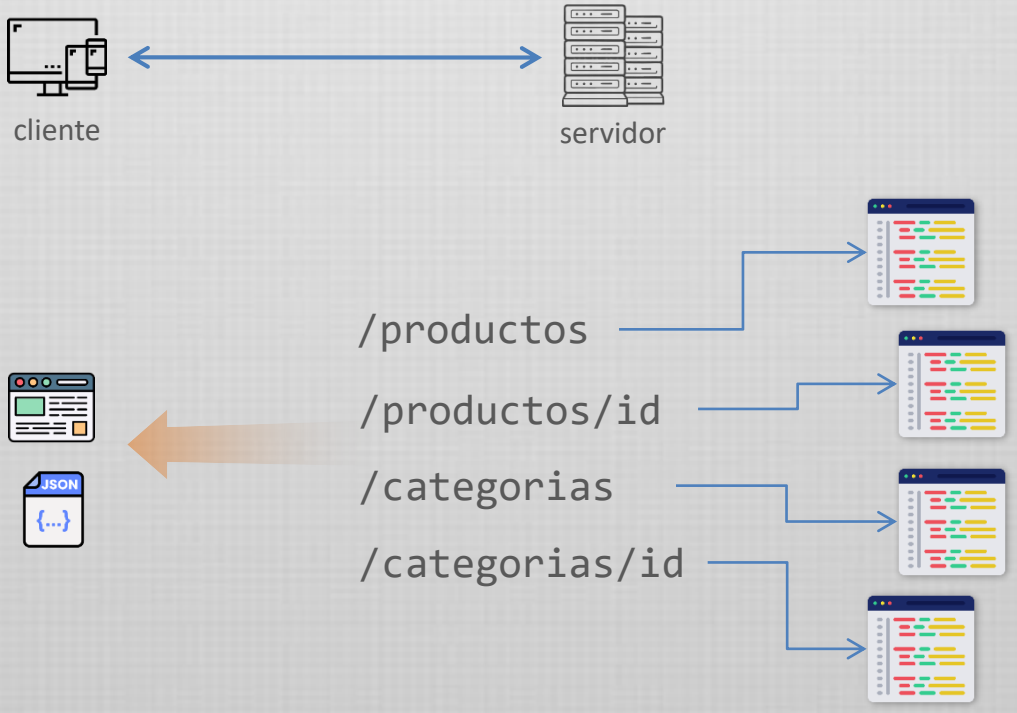
Operación CRUD	Método HTTP	Descripción
Create (Crear)	POST	Crea un nuevo recurso
Read (Leer)	GET	Obtiene uno o varios recursos
Update (Actualizar)	PUT o PATCH	Actualiza un recurso existente
Delete (Eliminar)	DELETE	Elimina un recurso

APIs basadas en recursos



CÓDIGO	NOMBRE	MÉTODO(S)	CUÁNDO USARLO
2XX — ÉXITO			
200	OK	GET PUT PATCH	Solicitud exitosa con cuerpo de respuesta. GET → devuelve el recurso. PUT/PATCH → devuelve el recurso actualizado.
201	Created	POST	Recurso creado exitosamente. Incluir `Location` header con la URL del nuevo recurso.
204	No Content	DELETE PUT PATCH	Operación exitosa sin cuerpo de respuesta. Típico en DELETE. También en PUT/PATCH si no se devuelve el recurso.

APIs


cliente
servidor
/productos
/productos/id
/categorias
/categorias/id
JSON
{...}

Ruteo

```
Route::get('/user', [UserController::class, 'index']);
```

```
<?php
$routes->get('users', 'Users::list’);
$routes->get('product/(:num)/(:num)', 'Product::index/$1/$2');
```

Ruteo

spring

```
PersonRepository repository = ...
PersonHandler handler = new PersonHandler(repository);

RouterFunction<ServerResponse> route = route()
        .GET("/person/{id}", accept(APPLICATION_JSON), handler::getPerson)
        .GET("/person", accept(APPLICATION_JSON), handler::listPeople)
        .POST("/person", handler::createPerson)
        .build();
```

```
public class PersonHandler {
        // ...
        public ServerResponse listPeople(ServerRequest request) { ... }
        public ServerResponse createPerson(ServerRequest request) { ... }
        public ServerResponse getPerson(ServerRequest request) { ... }
}
```

Ruteo

```
from django.urls import path

from . import views

urlpatterns = [
    path("articles/2003/", views.special_case_2003),
    path("articles/<int:year>/", views.year_archive),
    path("articles/<int:year>/<int:month>/", views.month_archive),
    path("articles/<int:year>/<int:month>/<slug:slug>/", views.article_detail),
]
```

Consideraciones de diseño de *paths*

Regla general

URLs = recursos (cosas)

NO ACCIONES

http://unsitio.com/producto/1/detalles

http://unsitio.com/vuelos/promociones

http://unsitio.com/inscripciones/evento/abc123

Consideraciones de diseño de *paths*

Comprensible por el humano

Fácil de identificar y memorizar

http://unsitio.com/promociones/2024

http://unsitio.com/2/3433123431321/xsd.1

Decidir los parámetros

¿Query Strings?

http://unsitio.com/abc?param1=zz&param2=32

¿Fragmentos del Path?

http://unsitio.com/abc/zz/32

Consideraciones de diseño de *paths*

Usar minúsculas

http://unsitio.com/abc?param1=zz&param2=32

http://unsitio.com/Argentina/Provincias

Aplicar jerarquías

http://unsitio.com/provincias/municipios/ciudades

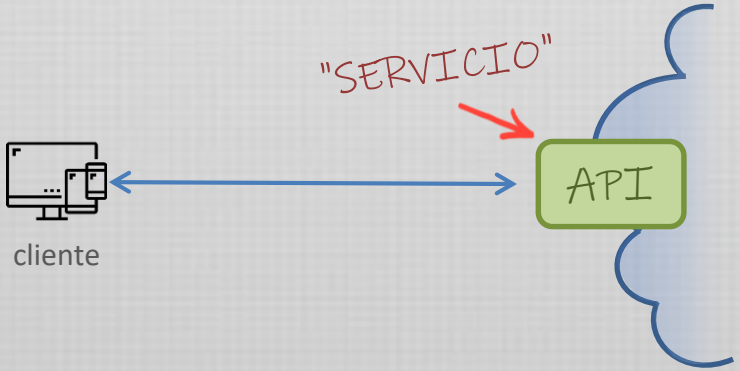
http://unsitio.com/productos/comestibles/organicos

http://unsitio.com/vehiculos/autos/electricos

¿Que características tiene que tener una “buena” API?

Tiene que ser **operacional**

La API hace lo que los usuarios necesitan



¿Que características tiene que tener una “buena” API?

Tiene que ser **expresiva**

Debe permitir expresar correctamente lo que se desea

traducir texto → `/traduccion/en/spa`

Encontrar hoteles → `/hotels/search`

¿Que características tiene que tener una “buena” API?

Tiene que ser **simple**

Apelar a nombres adecuados, no muy abstractos ni muy específicos

```
/plazoFijo
/plazoFijoTradicional
/plazoFijoUVA
/plazoFijoCancelable
```

```
/plazoFijo/tipo
```

¿Que características tiene que tener una “buena” API?

Tiene que ser **predecible**

Uniformidad en nombres y significados

`/productos/json`

↓

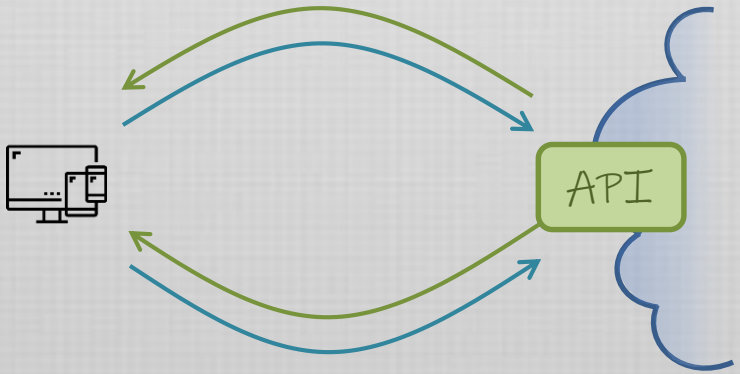
`/productos/xml`

`/producto/23/proveedor`

`/materias/7680/2026/1/inscriptos`

Existen algunos patrones de diseño habituales para APIs REST

STATELESS → Los requerimientos son independientes



Existen algunos patrones de diseño habituales para APIs REST

CONTENT NEGOTIATION → El formato de la respuesta es configurable (XML,JSON,JPG, PNG, etc)

GET http://localhost:9090/investors/invr_1/stocks/exa

Authorization | Headers (1) | Body | Pre-request Script | Tests

Key	Value
Accept	application/xml
New key	Value

Body | Cookies | Headers (4) | Test Results

Pretty | Raw | Preview | XML

```
<Stock>
    <symbol>EXA</symbol>
    <numberOfSharesHeld>200</numberOfSharesHeld>
    <tickerPrice>20.0</tickerPrice>
</Stock>
```

GET http://localhost:9090/investors/invr_1/stocks/exa

Authorization | Headers (1) | Body | Pre-request Script | Tests

Key	Value
Accept	application/json
New key	Value

Body | Cookies | Headers (4) | Test Results

Pretty | Raw | Preview | JSON

```
{
    "symbol": "EXA",
    "numberOfSharesHeld": 200,
    "tickerPrice": 20
}
```

Existen algunos patrones de diseño habituales para APIs REST

URI Templates → `http://mihost.com/{id_recurso1}/{id_recurso2}`

↓

`http://mihost.com/films/1980`

`http://mihost.com/actors/argentina`

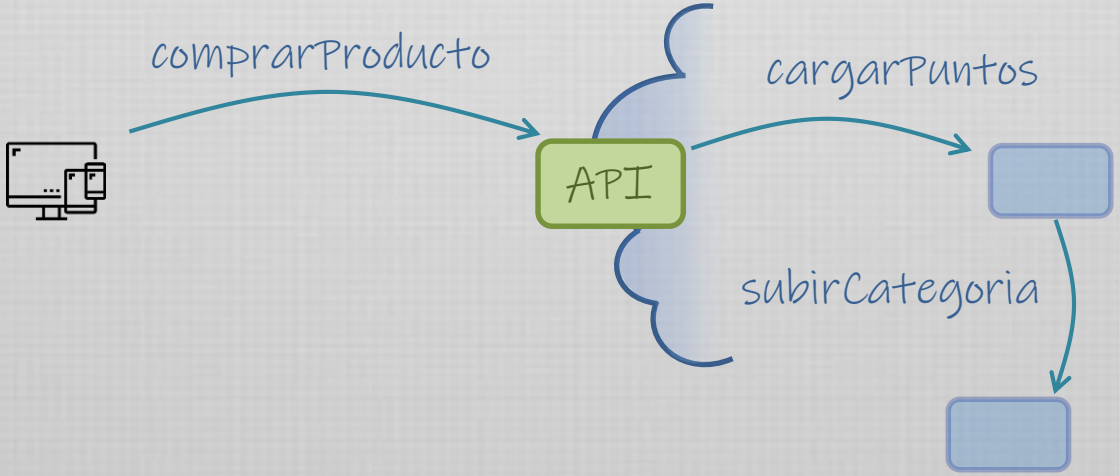
`http://mihost.com/investor/{id_inv}/stock/{symbol}`

↓

`http://mihost.com/investor/123/stock/BTC`

Existen algunos patrones de diseño habituales para APIs REST

DESIGN FOR INTENT → Diseñar para la intención del usuario, lo que puede demandar updates en varios recursos.



Existen algunos patrones de diseño habituales para APIs REST

PAGINATION → Ofrecer porciones de información

Por Offset

http://mihost.com/investor/123/stocks?offset=2&limit=5

Por Tiempo

http://mihost.com/investor/123/stocks?since=2010&until=2022

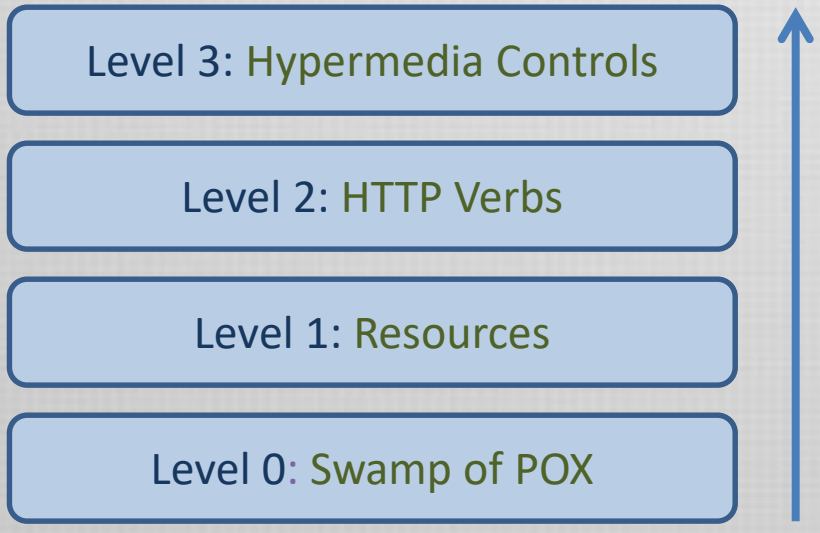
Por Cursor

http://mihost.com/estudiantes?size=500&before=ABCD

Richardson Maturity Model

Es una forma de calificar la API según las restricciones REST

Cuanto más adhiera el API a las restricciones, major calificación



Richardson Maturity Model

Es una forma de calificar la API según las restricciones REST

Cuanto más adhiera el API a las restricciones, major calificación

Level 3: Hypermedia Controls	Los datos contienen información sobre las posibles siguientes acciones (“links”)
Level 2: HTTP Verbs	Se utilizan los mensajes tipo CRUD
Level 1: Resources	Se identifican los recursos
Level 0: Swamp of POX	HTTP y *Plain Old Simple XML*

Level 3: Hypermedia Controls

Los datos contienen información sobre las posibles siguientes acciones (“links”)

http://www.autopartes.com/partes


```
{
  "partes": {
    "autoparte": [
      {
        "id": "ABC1",
        "href": "http://www.autopartes.com/partes/ABC1"
      },
      {
        "id": "DEF2",
        "href": "http://www.autopartes.com/partes/DEF2"
      },
      {
        "id": "GEW3",
        "href": "http://www.autopartes.com/partes/GEW3"
      },
      {
        "id": "KLM1",
        "href": "http://www.autopartes.com/partes/KLM1"
      }
    ]
  }
}
```

API como producto

La idea es considerar las APIs como un producto de software, no sólo como un componente técnico

La tendencia de construir clientes autónomos reafirma esta idea

Bases de API como producto

Centralizado en el cliente

Pensado según sus intereses: developers, product managers, ingenieros de seguridad, etc

Foco en el diseño de la API

Reusabilidad (utilidad en diversos tipos de aplicaciones)

Consistencia (uso predecible: convenciones de nombres, resultados, etc)

API Owner

Responsable del desarrollo y mantenimiento de la API

Tener en cuenta DX

Developer Experience – cómo los desarrolladores percibe y usan nuestra API

Estrategias de monetización

Dado que la API es en esencia un servicio de datos,
es frecuente la monetización

flujo rentable de ingresos directos o indirectos por el uso de la API por desarrolladores, socios y usuarios finales

Free

Sin costo
Fomenta la adopción de la API y su divulgación

Freemium

Acceso limitado
La limitación depende del servicio (tokens, hits, tiempo...)
Superado el límite hay que pagar

Pay-as-you-go

Se paga por cada consumo.
(ej. $0,0003 por hit)

Tiered

Se definen tipos de usos y beneficios, y se definen varias estructuras de precios

Estrategias de monetización

AccuWeather

Search Ctrl+K

Login Contact Us

Home Documentation Core Weather API MinuteCast™ API Lightning API Pricing FAQ

API Pricing

Core Weather MinuteCast™ Lightning

Core Weather	Free Trial	Starter	Standard	Prime	Elite
	Free 14 day trial 500 calls per day 1 key per developer **$0/mo** Start For Free	$0.25 CPM over 15.000 calls per month **$2/mo** Get started →	$0.12 CPM over 225.000 calls per month **$25/mo** Get started →	$0.15 CPM over 1.800.000 calls per month **$250/mo** Get started →	$0.22 CPM over 2.400.000 calls per month **$500/mo** Get started →
Locations	✓	✓	✓	✓	✓
Current Conditions	✓	✓	✓	✓	✓
24 Hours Historical Current Conditions	✓		✓	✓	✓
Daily Forecast	5 Days	5 Days	5 Days	10 Days	15 Days
Hourly Forecast	12 Hours	12 Hours	12 Hours	72 Hours	120 Hours

Estrategias de monetización

OpenAI | Investigación | Productos | Negocios | Desarrolladores | Empresa | Fundación | Iniciar sesión | Probar ChatGPT ↗

GPT-5.4

Nuestro modelo más competente para el trabajo profesional.

Precio

Entrada:
USD 2.50/1 millón de tokens

Entrada en caché:
USD 0.25/1 millón de tokens

Salida:
USD 15.00/1 millón de tokens

GPT-5.4 mini

Nuestro modelo mini más potente hasta la fecha para programación, uso de computadoras y subagentes.

Precio

Entrada:
USD 0.75/1 millón de tokens

Entrada en caché:
USD 0.075/1 millón de tokens

Salida:
USD 4.50/1 millón de tokens

GPT-5.4 nano

Nuestro modelo más económico de la gama GPT-5.4 para tareas sencillas de gran volumen.

Precio

Entrada:
USD 0.20/1 millón de tokens

Entrada en caché:
USD 0.02/1 millón de tokens

Salida:
USD 1.25/1 millón de tokens

Los precios anteriores corresponden a las tarifas de procesamiento estándar para longitudes de contexto menores a 270 000.

Obtener más información sobre: Procesamiento por lotes y Residencia de datos y Procesamiento regional

Estrategias de monetización

CoinAPI.io by API BRICKS

Products | Use Cases | Enterprise | Pricing | Developers | Resources | Contact us | Log in

Market Data API

Overview | Documentation | Exchanges & Symbols | Data Samples | Use Cases | Pricing | FAQ | Tutorials

PAY AS YOU GO	STARTUP	STREAMER	PRO	ENTERPRISE
Metered monthly	$79 monthly	$249 monthly	$599 monthly	Custom monthly
Try for free	Subscribe	Subscribe	Subscribe	Let's talk
• $25.00 Free Credits • Historical & Real-time Data • REST Protocol • WebSocket Protocol • Volume Discounts • Email Support	• All from Metered Package • 1k REST Credits/Day • 32 GB of Tier 1 Data/Day	• All from Startup Package • 10k REST Credits/Day • 128 GB of Tier 1 Data/Day • 1 GB of Tier 2 Data/Day	• All from Streamer Package • 100k REST Credits/Day • 512 GB of Tier 1 Data/Day • 64 GB of Tier 2 Data/Day • FIX API • Premium Support	• All Features of Lower Packages • Custom Limits & Quotas • Project Tailored SLAs • Premium 24/7 Support • Latency SLA • Bundle options with other